

Министерство образования Республики Беларусь
Учреждение образования
“БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ”

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Методы защиты информации

ОТЧЁТ
к лабораторной работе 6
на тему

Цифровая подпись

Выполнил

Т.П. Власенко

Проверил

А.В. Герчик

Минск 2024

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Теоретические сведения	4
3 Демонстрация работы	9
Выводы	10
Приложение А (обязательное) Исходный код программы	11

1 ПОСТАНОВКА ЗАДАЧИ

Реализовать программное средство формирования и проверки ЭЦП на базе алгоритма ГОСТ 34.10.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

ГОСТ 34.10-2018 (полное название: «ГОСТ 34.10-2018. Информационная технология. Криптографическая защита информации. Процессы формирования и проверки электронной цифровой подписи», англ. «Information technology. Cryptographic data security. Signature and verification processes of electronic digital signature») — действующий межгосударственный криптографический стандарт, описывающий алгоритмы формирования и проверки электронной цифровой подписи, реализуемой с использованием операций в группе точек эллиптической кривой, определенной над конечным простым полем.

Стандарт разработан на основе национального стандарта Российской Федерации ГОСТ Р 34.10-2012 и введен в действие с 1 июня 2019 года приказом Росстандарта № 1059-ст от 4 декабря 2018 года.

Данный алгоритм не описывает механизм генерации параметров, необходимых для формирования подписи, а только определяет, каким образом на основании таких параметров получить цифровую подпись. Механизм генерации параметров определяется на месте в зависимости от разрабатываемой системы.

ГОСТ 34.10:

Формирование цифровой подписи:

Для получения цифровой подписи под сообщением необходимо выполнить следующие действия (шаги) по алгоритму:

Шаг 1 — вычислить хэш-код сообщения.

Шаг 2 — вычислит целое число a , двоичным представлением которого является вектор h , и определить e . Если $e = 0$, то определить $e = 1$.

Шаг 3 — сгенерировать случайное (псевдослучайное) целое число k , удовлетворяющее неравенству.

Шаг 4 — вычислить точку эллиптической кривой $C = kP$ и определить r . Если $r = 0$, то необходимо вернуться к шагу 3.

Шаг 5 — вычислить значение $s = (rd + ke)(\text{mod } q)$. Если $s = 0$, то необходимо вернуться к шагу 3.

Шаг 6 — вычислить двоичные векторы r' и s' , соответствующие r и s , и определить цифровую подпись $C = (r' \parallel s')$ как конкатенацию двух двоичных векторов.

Исходными данными этого процесса являются ключ подписи d и подписываемое сообщение M , а выходным результатом — цифровая подпись C .

Схема процесса формирования цифровой подписи приведена на рисунке 1.

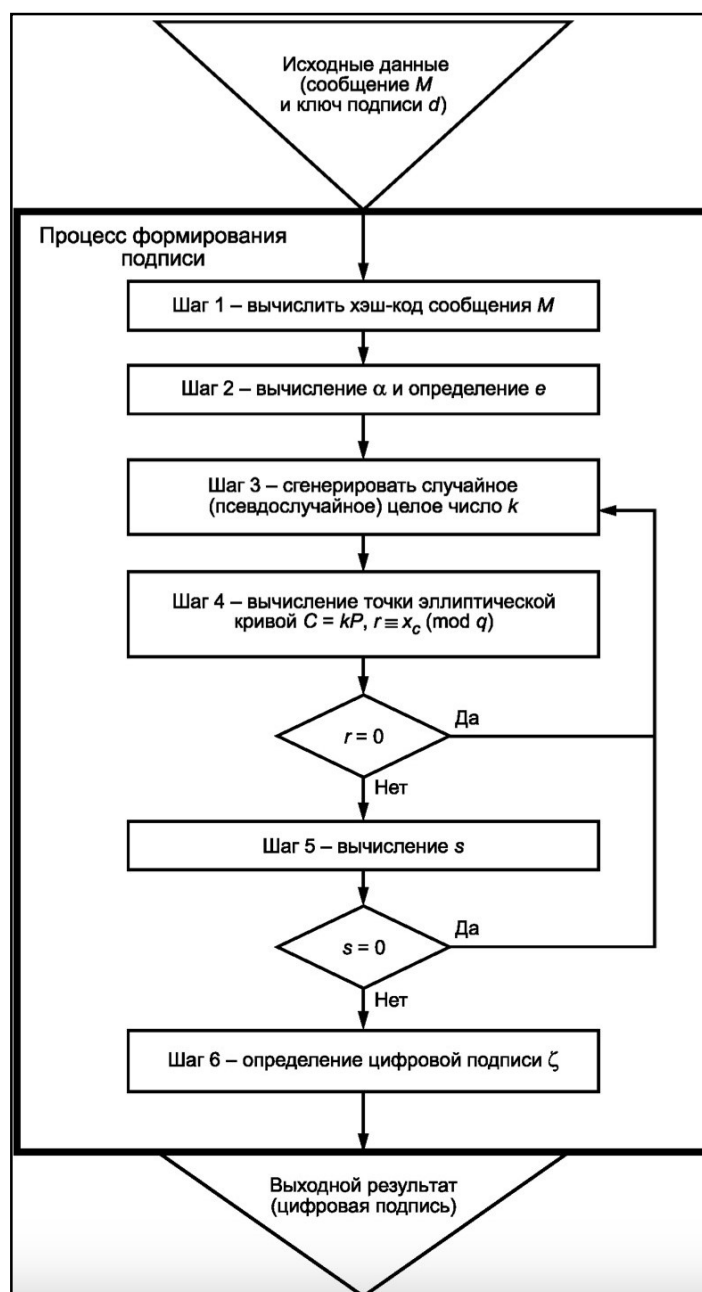


Рисунок 1 - Схема процесса формирования цифровой подписи

Проверка цифровой подписи:

Для проверки цифровой подписи S под полученным сообщением M необходимо выполнить следующие действия (шаги) по алгоритму II:

Шаг 1 — по полученной подписи S вычислить целые числа r и s . Если выполнены неравенства $0 < r < q$, $0 < s < q$, то перейти к следующему шагу. В противном случае подпись неверна.

Шаг 2 — вычислить хэш-код полученного сообщения M $h' = h(M)$.

Шаг 3 — вычислить целое число a , двоичным представлением которого является вектор h , и определить e . Если $e = 0$, то определить $e = 1$.

Шаг 4 — вычислить значение $v = e^{-1} \pmod{q}$.

Шаг 5 — вычислить значения z_1, z_2 .

Шаг 6 — вычислить точку эллиптической кривой $C = z_1P + z_2Q$ и определить R .

Шаг 7 — если выполнено равенство $R = r$, то подпись принимается, в противном случае — подпись неверна.

Исходными данными этого процесса являются подписанное сообщение M , цифровая подпись S , и ключ проверки подписи Q , а выходным результатом — свидетельство о достоверности или ошибочности данной подписи.

Схема процесса проверки цифровой подписи приведена на рисунке 2.

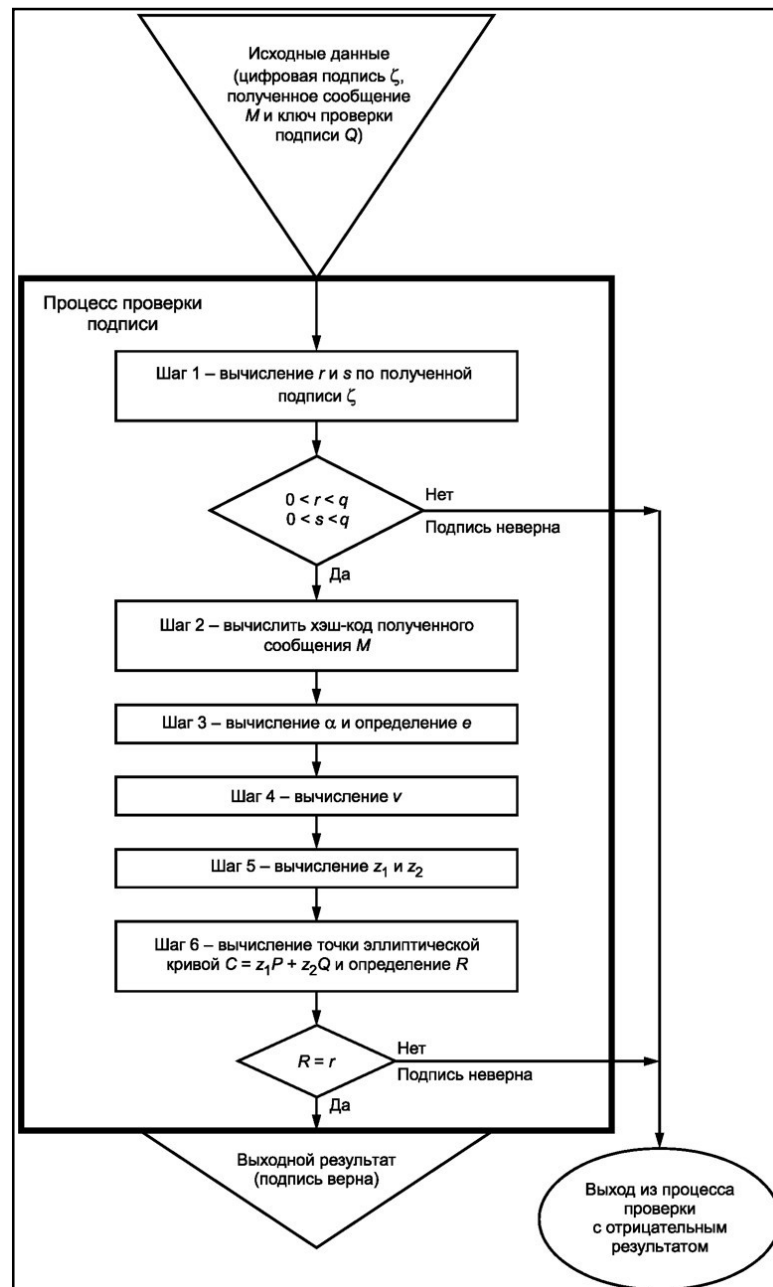


Рисунок 2 - Схема проверки цифровой подписи

3 ДЕМОНСТРАЦИЯ РАБОТЫ

Пример работы программы показан на рисунке 3.

```
Message: привет world 123 #$$%^  
Signature:  
37681d7f80bd1c4f0e7fff0f305a180e56b3459108f02608125a7444763ada88  
363cb0c1959c5e9669cf313e0f2dbb3de10308565991f56e45400af4048e7c9f  
Correct signature
```

Рисунок 3 – Пример работы программы

ВЫВОДЫ

В ходе выполнения данной лабораторной работы было реализовано программное средство контроля цифровой подписи с помощью вычисления хэш-функции ГОСТ 34.10.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

```
import hashlib
import random

def point_add(a, b, p, P1, P2):
    x1, y1 = P1
    x2, y2 = P2

    if (x1, y1) == (0, 0):
        return P2
    if (x2, y2) == (0, 0):
        return P1
    if x1 == x2 and (y1 != y2 or y1 == 0):
        return (0, 0)

    if x1 == x2 and y1 == y2:
        m = (3 * x1 ** 2 + a) * multiplicative_inverse(2 * y1, p) % p
    else:
        m = (y2 - y1) * multiplicative_inverse(x2 - x1, p) % p

    x3 = (m ** 2 - x1 - x2) % p
    y3 = (m * (x1 - x3) - y1) % p
    return (x3, y3)

def point_multiply(a, b, p, P, n):
    result = (0, 0)
    addend = P

    for bit in bits(n):
        if bit == 1:
            result = point_add(a, b, p, result, addend)
            addend = point_add(a, b, p, addend, addend)
    return result

def bits(number):
    while number:
        yield number & 1
        number >>= 1

def euc_ext(a, b):
    x, x_, y, y_ = 1, 0, 0, 1
    while b:
        q = a // b
        a, b = b, a % b
        x, x_ = x_, x - x_ * q
        y, y_ = y_, y - y_ * q
    return x, y

def multiplicative_inverse(a, b):
    x, _ = euc_ext(a, b)
    return (x + b) % b
```



```
print(f"{signature[1]:x}")  
print("Correct signature" if is_correct else "Wrong signature")
```